

רשתות זיכרון ארוך-קצר: מדריך מקיף ל-LSTM

Long Short-Term Memory Networks: A Comprehensive Guide to LSTM

ד"ר סגל יורם

כל הזכויות שמורות - © ד"ר סגל יורם

January 2025

תקציר

ספר זה מציג מבוא מקיף לרשתות LSTM (Long Short-Term Memory), אחד המנגנונים המרכזיים בעיבוד רצפים ולמידה עמוקה. הספר מסביר את הבעיות ב-RNN רגיל, את הפתרון שמציע LSTM, את המבנה הפנימי עם שלושת השערים, ואת הסיבות שבגללן LSTM עובד כל כך טוב.

הספר מיועד לסטודנטים ומעשים בתחום הלמידה העמוקה שרוצים להבין לעומק את LSTM, עם הסברים תיאורטיים, נוסחאות מתמטיות, ודוגמאות מעשיות.

תוכן העניינים

4		The Problem LSTM Solves פותרת:	1
4		Quick RNN Recap :RNN סיכום מהיר של	1.1
4		The Gradient Crisis משבר הגרדיאנט:	2.1
5		What We Need מה אנחנו צריכים:	3.1
6		LSTM Core Idea :LSTM הרעיון המרכזי של	2
6		The Cell State Concept מושג מצב התא:	1.2
6		Two States Instead of One שני מצבים במקום אחד:	2.2
7		Gates Control Flow שערים שולטים בזרימה:	3.2
8		The Three Gates שלושת השערים:	3
8		Forget Gate שער שכחה:	1.3
8		Input Gate שער כניסה:	2.3
9		Output Gate שער פלט:	3.3
01		סיכום: השערים פועלים עצמאית	4.3
11		Information Flow Step-by-Step זרימת מידע צעד-אחר-צעד:	4
11		שלב 1: החלטה מה לשכוח (Decide What to Forget)	1.4
11		שלב 2: החלטה מה להוסיף (Decide What to Add)	2.4
21		שלב 3: עדכון מצב התא (Update Cell State)	3.4
21		שלב 4: יצירת פלט (Generate Output)	4.4
31		סיכום: רצף הפעולות המלא	5.4
41		Why LSTM Works למה LSTM עובד:	5
41		The Gradient Highway כביש הגרדיאנט:	1.5
41		Selective Memory זיכרון סלקטיבי:	2.5
51		Long-Term Dependencies תלויות לטווח ארוך:	3.5
51		Training Benefits יתרונות באימון:	4.5
51		Limitations מגבלות:	5.5
61		Comparisons השוואות:	6.5
71		Appendix: LSTM Equations Summary :LSTM סיכום משוואות	6
71		All Equations in One Place כל המשוואות במקום אחד:	1.6
81		Notation Guide מילון סימונים:	2.6
81		Hyperparameter Tuning Tips טיפים לכוונון היפר-פרמטרים:	3.6
81		Further Reading קריאה נוספת:	4.6

English References19

1 הבעיה ש-LSTM פותרת: The Problem LSTM Solves

1.1 סיכום מהיר של RNN: Quick RNN Recap

רשתות נוירונים חוזרות (Recurrent Neural Networks, RNN) מעבדות נתונים סדרתיים באמצעות לולאות משוב שמעבירות מידע בין צעדי זמן. המשמעות היא שהרשת "זוכרת" מידע מצעדים קודמים. במבנה RNN רגיל, משקלים משותפים על פני הזמן מאפשרים למודל ללמוד דפוסים חוזרים בנתונים.

מבנה RNN בסיסי כולל:

- עיבוד נתונים סדרתיים: כל כניסה x_t מעובדת בתור
 - לולאות משוב: המצב המוסתר h_{t-1} עובר לצעד הבא
 - שיתוף משקלים: אותן מטריצות משקלים W בכל צעד זמן
- המשוואה הבסיסית של RNN:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h) \quad (1.1)$$

1.2 משבר הגרדיאנט: The Gradient Crisis

כאשר אנו מנסים לאמן RNN על רצפים ארוכים, אנו נתקלים בבעיה קריטית: **גרדיאנטים נעלמים או מתפוצצים** (Vanishing/Exploding Gradients). בשנת 1994, Bengio et al. [1] הוכיחו מתמטית שקשה לאמן RNN רגיל ללמוד תלויות לטווח ארוך.

הבעיות העיקריות:

1. **רצפים ארוכים → גרדיאנטים נעלמים:** כאשר n גדל (מספר צעדי זמן), הגרדיאנט קטן באופן אקספוננציאלי
2. **מידע הולך לאיבוד או מעוות:** מידע מצעדים רחוקים נמחק
3. **אי-יכולת ללמוד תלויות לטווח ארוך:** הרשת לא יכולה "לזכור" מידע מ-50 צעדים אחורה
4. **אימון הופך לבלתי אפשרי:** הלמידה נעצרת או הופכת לא יציבה

הסבר מתמטי לנעלמות הגרדיאנט:

בחזרה אחורה דרך הזמן (Backpropagation Through Time, BPTT), הגרדיאנט נמשך כך:

$$\frac{\partial h_t}{\partial h_{t-n}} = \prod_{i=1}^n \frac{\partial h_{t-i+1}}{\partial h_{t-i}} = \prod_{i=1}^n W_{hh}^T \cdot \text{diag}(\tanh'(z_{t-i})) \quad (1.2)$$

כאשר n גדל, המכפלה הזו גורמת ל:

- **נעלמות:** אם $\|W_{hh}\| < 1$, אז $(W_{hh})^n \rightarrow 0$ כש- $n \rightarrow \infty$
- **התפוצצות:** אם $\|W_{hh}\| > 1$, אז $(W_{hh})^n \rightarrow \infty$ כש- $n \rightarrow \infty$

פירוק ריקוד של הגרדיאנט:

$$\frac{\partial \mathcal{L}}{\partial W} \propto W^n \rightarrow 0 \quad (1.3)$$

כאשר n גדל

דוגמה מעשית:

נניח שאנו מעבדים משפט: "הילד שלמד צרפתית במשך 10 שנים דיבר בצרפתית".
הרשת צריכה "לזכור" את המילה "הילד" כדי לנבא נכון "דיבר" (זכר, יחיד). ב-RNN רגיל, המידע על "הילד" נמחק לאחר 10 מילים.

1.3 מה אנחנו צריכים: What We Need

כדי לפתור את הבעיה, אנו זקוקים למנגנון שיכול:

1. **לזכור מידע חשוב לטווח ארוך:** לשמור מידע רלוונטי למאות צעדים
2. **לשכוח מידע לא רלוונטי:** למחוק מידע ישן שאינו נחוץ
3. **לשלוט בזרימת המידע:** להחליט מה להוסיף ומה להוציא
4. **גרדיאנטים יציבים:** להימנע מנעלמות/התפוצצות גרדיאנטים

דרישות טכניות:

- זיכרון סלקטיבי: לא כל המידע שווה חשיבות
 - מסלול גרדיאנט ישיר: דרך קצרה לזרימת גרדיאנטים
 - שערים לומדים: מנגנונים שמחליטים מה לעשות עם המידע
- כל אלה מובילים לפתרון המהפכני: **Long Short-Term Memory (LSTM)**.

2 הרעיון המרכזי של LSTM: LSTM Core Idea

2.1 מושג מצב התא: The Cell State Concept

החידוש המרכזי ב-LSTM [2] הוא **מצב התא** (Cell State, מסומן C_t). זהו "כביש מידע" שעובר ישירות דרך הזמן עם שינויים מינימליים. תכונות מצב התא:

- **נפרד ממצב מוסתר:** C_t הוא ישות עצמאית
 - **רץ ישירות בזמן:** קו אופקי במבנה LSTM
 - **שינויים מינימליים:** רק פעולות לינאריות עליו
 - **כביש מידע:** מאפשר זרימת מידע ללא הפסדים
- המשמעות: מידע יכול "לזרום" מצעד 0 לצעד 100 בלי להישחק!

2.2 שני מצבים במקום אחד: Two States Instead of One

LSTM משתמש בשני מצבים במקום מצב אחד:

1. מצב התא (C_t): זיכרון לטווח ארוך (Long-Term Memory)

- שומר מידע חשוב לאורך זמן
- מתעדכן בזהירות דרך שערים
- דומה ל"דיסק קשיח" של המוח

2. מצב מוסתר (h_t): זיכרון לטווח קצר/פלט (Short-Term Memory)

- מכיל מידע רלוונטי מיידי
- משמש כפלט של התא
- דומה ל-RAM של המוח

שני המצבים זורמים קדימה בזמן ופועלים ביחד:

$$C_{t-1} \xrightarrow{\text{updates}} C_t \xrightarrow{\text{next}} C_{t+1} \quad (2.1)$$

$$h_{t-1} \xrightarrow{\text{computation}} h_t \xrightarrow{\text{next}} h_{t+1} \quad (2.2)$$

2.3 שערות שולטים בזרימה: Gates Control Flow

השערות (Gates) הם המנגנון המרכזי ב-LSTM. כל שער הוא מסנן שמחליט מה עובר דרכו. מהו שער? שער הוא פונקציה שמחזירה ערכים בין 0 ל-1:

$$\text{gate}(x) = \sigma(W \cdot x + b) \quad (2.3)$$

כאשר:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

משמעות הערכים:

- $\text{gate} = 0 \rightarrow$ חוסם לחלוטין (שער סגור)
- $\text{gate} = 0.5 \rightarrow$ חלקי (שער חצי פתוח)
- $\text{gate} = 1 \rightarrow$ מעביר הכל (שער פתוח לגמרי)

ב-LSTM יש שלושה שערים:

1. שער שכחה (Forget Gate): מחליט מה למחוק
2. שער כניסה (Input Gate): מחליט מה להוסיף
3. שער פלט (Output Gate): מחליט מה להוציא

כל השערים נלמדים במהלך האימון באמצעות backpropagation.

דוגמה אינטואיטיבית:

דמיינו שאתם קוראים ספר ומתעדים הערות:

- שער שכחה: "האם מחוק את ההערה הישנה הזו?"
- שער כניסה: "האם להוסיף הערה חדשה זו?"
- שער פלט: "האם להשתמש בהערה זו עכשיו?"

המשוואה הכללית לשער:

$$\text{gate}_t = \sigma(W_{\text{gate}} \cdot [h_{t-1}, x_t] + b_{\text{gate}}) \quad (2.4)$$

כאשר:

- h_{t-1} : מצב מוסתר קודם
- x_t : כניסה נוכחית
- $[h_{t-1}, x_t]$: שרשור (concatenation) של שני הוקטורים
- W_{gate} : מטריצת משקלים לומדת
- b_{gate} : וקטור הטיה (bias)

3 שלושת השערים: The Three Gates

3.1 שער שכחה: Forget Gate

תפקיד: מחליט איזה מידע להסיר ממצב התא.

שער השכחה קורא את h_{t-1} (מצב מוסתר קודם) ואת x_t (כניסה נוכחית), ומחזיר ערכים בין 0 ל-1 עבור כל רכיב במצב התא C_{t-1} .

המשוואה:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (3.1)$$

פרשנות:

- $f_t = 0 \rightarrow$ "שכח לחלוטין" (מחק את המידע)

- $f_t = 1 \rightarrow$ "שמור לחלוטין" (שמור את כל המידע)

- $f_t = 0.7 \rightarrow$ "שמור 70% מהמידע"

דוגמה: במשפט "הילדה אוכלת גלידה" הוא שותה מיץ, כשמגיעים ל"הוא", שער השכחה מחליט לשכוח את הנושא "הילדה" כי מתחיל נושא חדש.

3.2 שער כניסה: Input Gate

תפקיד: מחליט איזה מידע חדש להוסיף למצב התא.

שער הכניסה פועל בשני חלקים:

1. **מסנן (שער):** מחליט אילו ערכים לעדכן

2. **ערכי מועמד:** מחשב אילו ערכים חדשים להוסיף

משוואות:

מסנן (שער):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (3.2)$$

ערכי מועמד:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (3.3)$$

הבדל בין σ ל- $\tanh$:

- σ (sigmoid): טווח (0, 1) - משמש למסננים

- $\tanh$: טווח (-1, 1) - משמש לערכים ממויינים

השילוב של שני החלקים (מידע חדש):

$$i_t \odot \tilde{C}_t \quad (3.4)$$

כאשר $\odot$ הוא כפל איבר-איבר (element-wise multiplication).

דוגמה: כשרואים "הוא שותה", שער הכניסה מחליט להוסיף מידע חדש: "נושא: זכר, פועל: שותה".

3.3 שער פלט: Output Gate

תפקיד: מחליט מה להוציא כפלט (המצב המוסתר h_t).

שער הפלט מסנן את מצב התא C_t ליצירת המצב המוסתר h_t שמועבר הלאה.

משוואות:

שער פלט:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (3.5)$$

מצב מוסתר:

$$h_t = o_t \odot \tanh(C_t) \quad (3.6)$$

למה $\tanh(C_t)$? $\tanh$ ממפה את C_t לטווח (-1, 1), מה שמנרמל את הערכים ומסייע ליציבות המספרית.

פרשנות: שער הפלט שולט ב"מה הרשת רוצה לומר" בצעד הזה. הוא מחליט אילו חלקים של הזיכרון לטווח ארוך (C_t) רלוונטיים לצעד הנוכחי.

דוגמה: כשמנבאים את המילה הבאה אחרי "הוא שותה", שער הפלט מחליט להתמקד במידע על "משקאות" ולא על נושאים קודמים.

3.4 סיכום: השערים פועלים עצמאית

חשוב להבין שכל שער:

- משתמש ב-diomgis (σ) לייצור ערכים בין 0 ל-1
 - נלמד בנפרד במהלך האימון
 - פועל באופן עצמאי - החלטה של שער אחד לא משפיעה ישירות על אחר
 - משתמש במשקלים שונים (W_C, W_o, W_i, W_f)
- טבלת סיכום:

טבלה 1: סיכום שלושת השערים: Summary of Three Gates

שער / Gate	שאלה / Question	פונקציה / Function
שכחה / Forget	מה לשכוח? / What to forget?	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
כניסה / Input	מה לזכור? / What to remember?	$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
פלט / Output	מה להוציא? / What to output?	$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

4 זרימת מידע צעד-אחר-צעד: Information Flow Step-by-Step

בפרק זה נעקוב אחר הנתונים דרך צעד זמן אחד ב-LSTM, משלב לשלב.

4.1 שלב 1: החלטה מה לשכוח (Decide What to Forget)

הפעולה הראשונה: שער השכחה מופעל ומחליט מה למחוק ממצב התא הקודם.

צעדים:

1. חישוב שער השכחה: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

2. כפל עם מצב התא הקודם: $C_{t-1} \odot f_t$

3. הסרת מידע לא רצוי

דוגמה מספרית: נניח $C_{t-1} = [0.8, -0.5, 0.3, 0.9]$ ושער השכחה החזיר $f_t = [1.0, 0.0, 0.5, 1.0]$.
התוצאה:

$$C_{t-1} \odot f_t = [0.8 \times 1.0, -0.5 \times 0.0, 0.3 \times 0.5, 0.9 \times 1.0]$$

$$= [0.8, 0.0, 0.15, 0.9] \quad (4.1)$$

הרכיב השני נמחק לחלוטין!

4.2 שלב 2: החלטה מה להוסיף (Decide What to Add)

הפעולה השנייה: יצירת מידע חדש והחלטה מה להוסיף למצב התא.

צעדים:

1. חישוב שער כניסה (מסנן): $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$

2. יצירת ערכי מועמד: $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$

3. כפל מסנן בערכי מועמד: $i_t \odot \tilde{C}_t$

דוגמה מספרית: נניח $\tilde{C}_t = [0.6, 0.9, -0.4, 0.2]$ ושער הכניסה החזיר $i_t = [0.8, 0.0, 1.0, 0.3]$.
התוצאה:

$$i_t \odot \tilde{C}_t = [0.8 \times 0.6, 0.0 \times 0.9, 1.0 \times (-0.4), 0.3 \times 0.2]$$

$$= [0.48, 0.0, -0.4, 0.06] \quad (4.2)$$

4.3 שלב 3: עדכון מצב התא (Update Cell State)

הפעולה השלישית: שילוב פעולות השכחה וההוספה ליצירת מצב תא חדש.
המשוואה המרכזית:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (4.3)$$

זו המשוואה החשובה ביותר ב-LSTM!

דוגמה מספרית (המשך): משלב 1: $f_t \odot C_{t-1} = [0.8, 0.0, 0.15, 0.9]$

משלב 2: $i_t \odot \tilde{C}_t = [0.48, 0.0, -0.4, 0.06]$

עדכון:

$$C_t = [0.8 + 0.48, 0.0 + 0.0, 0.15 + (-0.4), 0.9 + 0.06]$$

$$C_t = [1.28, 0.0, -0.25, 0.96] \quad (4.4)$$

תובנה חשובה: שימו לב שהפעולה היא **חיבור**, לא כפל! זה מאפשר לגרדיאנט לזרום בקלות.

4.4 שלב 4: יצירת פלט (Generate Output)

הפעולה הרביעית: הפקת המצב המוסתר h_t שמועבר לצעד הבא ומשמש כפלט.

צעדים:

1. חישוב שער פלט: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

2. נרמול מצב התא: $\tanh(C_t)$

3. סינון לפי שער הפלט: $h_t = o_t \odot \tanh(C_t)$

דוגמה מספרית (המשך): $C_t = [1.28, 0.0, -0.25, 0.96]$

נרמול:

$$\tanh(C_t) = [\tanh(1.28), \tanh(0.0), \tanh(-0.25), \tanh(0.96)]$$

$$\approx [0.86, 0.0, -0.24, 0.74] \quad (4.5)$$

נניח שער הפלט: $o_t = [1.0, 0.5, 0.0, 1.0]$

פלט סופי:

$$h_t = o_t \odot \tanh(C_t) = [0.86, 0.0, 0.0, 0.74] \quad (4.6)$$

המצב המוסתר h_t זה:

- עובר לצעד הבא $t + 1$

- משמש כפלט של התא

- משפיע על חיזוי המילה/ערך הבא

4.5 סיכום: רצף הפעולות המלא

1. **שכחה:** $f_t \odot C_{t-1}$ - מחיקת מידע ישן

2. **הוספה:** $i_t \odot \tilde{C}_t$ - הוספת מידע חדש

3. **עדכון:** $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ - מצב תא חדש

4. **פלט:** $h_t = o_t \odot \tanh(C_t)$ - מצב מוסתר חדש

הערה חשובה: הסימון $\odot$ פירושו כפל איבר-איבר, לא כפל מטריצות!

כל הפעולות האלה קורות **בכל צעד זמן**, ומאפשרות ל-LSTM לשלוט בדיוק במידע שהוא שומר, מוסיף, ומוציא.

5 למה LSTM עובד: Why LSTM Works

5.1 כביש הגרדיאנט: The Gradient Highway

הסיבה המרכזית שבגללה LSTM פותר את בעיית הגרדיאנטים הנעלמים היא **מצב התא כ"כביש מידע"**.

ההבדל המכריע:

- **ב-RNN:** הגרדיאנט עובר דרך כפל חוזר ונשנה של W

$$\frac{\partial h_t}{\partial h_{t-n}} = \prod_{i=1}^n W^T \rightarrow 0 \quad (5.1)$$

או ∞

- **ב-LSTM:** הגרדיאנט זורם דרך **חיבור**, לא כפל

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t \quad (5.2)$$

(ערך בין 0 ל-1, לא חזקה!)

למה חיבור עדיף על כפל?

בחזרה אחורה דרך n צעדים:

$$\frac{\partial C_t}{\partial C_{t-n}} = \prod_{i=1}^n f_{t-i+1} \quad (5.3)$$

אבל f_t משתנה בכל צעד (לא קבוע כמו W), ולכן:

- אין ריקבון/התפוצצות אקספוננציאלית
- הגרדיאנט יכול לזרום בקלות +100 צעדים אחורה
- הרשת יכולה ללמוד תלויות לטווח ארוך

5.2 זיכרון סלקטיבי: Selective Memory

היתרון השני הוא שהשערים **לומדים** מה חשוב ומה לא.

לעומת משקלים קבועים:

- **RNN רגיל:** משקלים קבועים W לכל צעד $\rightarrow$ ללא גמישות

- **LSTM:** שערים דינמיים $f_t, i_t, o_t \rightarrow$ החלטות מותאמות

דוגמה: במשפט "הילדה, שגרה בתל-אביב ולומדת מדעי המחשב, אוכלת גלידה", LSTM לומד:

1. לשמור "הילדה" (נושא חשוב)
2. לשכוח "גרה בתל-אביב, לומדת מדעי המחשב" (פרטים לא רלוונטיים לפועל)

3. להוציא "נקבה, יחיד" לניבוי "אוכלת"

כל זה נלמד אוטומטית מהנתונים!

5.3 תלויות לטווח ארוך: Long-Term Dependencies

LSTM יכול לזכור מידע למאות צעדים:

ביצועים:

- יכול לזכור הקשר מ-100+ צעדים אחורה [3]
- הקשר נשמר על פני הזמן ללא הידרדרות
- מאפשר הבנת משפטים מורכבים, סיפורים, דיאלוגים

דוגמאות יישומים:

1. **עיבוד שפה:** "tac ehT ,tac hcihw ,...eta ydaerla lluf saw" הרשת זוכרת "tac" כדי לנבא "saw" (לא "erew")
2. **סדרות זמן:** ניבוי מחיר מניה מבוסס על מגמה מ-60 ימים אחורה
3. **דיבור:** זיהוי הקשר של משפט שלם בקובץ אודיו ארוך

5.4 יתרונות באימון: Training Benefits

LSTM לא רק פותר בעיות, אלא גם מאמן טוב יותר:

1. **גרדיאנטים יציבים:** ללא נעלמות/התפוצצות
2. **התכנסות מהירה יותר:** הלמידה יעילה יותר
3. **ביצועים טובים יותר:** דיוק גבוה יותר במשימות
4. **תקן תעשייתי:** לפני Transformers, LSTM היה התקן מספר פרמטרים: LSTM דורש פי 4 יותר פרמטרים מ-RNN רגיל:

$$\text{Params} = 4 \times (d_{\text{input}} \times d_{\text{hidden}} + d_{\text{hidden}}^2 + d_{\text{hidden}}) \quad (5.4)$$

למשל, $d_{\text{hidden}} = 128$, $d_{\text{input}} = 100$

$$\text{Params} = 4 \times (100 \times 128 + 128^2 + 128) = 4 \times 29,056 = 116,224 \quad (5.5)$$

5.5 מגבלות: Limitations

למרות היתרונות, ל-LSTM יש מגבלות:

1. **עיבוד סדרתי:** לא ניתן למקבל לחלוטין $\rightarrow$ אימון איטי
2. **איטי יותר משיטות מקבילות:** Transformers מהירים יותר
3. **יותר פרמטרים:** $4 \times$ יותר מ-RNN, צורך בזיכרון
4. **Transformers מועדפים כיום:** ב-NLP, Attention מחליף LSTM

מתי להשתמש ב-LSTM?

- כאשר נתונים סדרתיים קצרים יחסית (פחות מ-500 צעדים)
- כאשר זיכרון UPG מוגבל
- בסדרות זמן (מחירי מניות, חיישנים, אותות)
- כשצריך עיבוד בזמן אמת (streaming)

5.6 השוואות: Comparisons

LSTM לעומת GRU: GRU (Gated Recurrent Unit) [4] הוא גרסה פשוטה יותר:

- רק 2 שערים (במקום 3)
- ללא מצב תא נפרד
- מהיר יותר לאימון
- ביצועים דומים ברוב המקרים

LSTM לעומת Transformers: Transformers [5] משתמשים במנגנון קשב (Attention):

- עיבוד מקבילי לחלוטין
- מהירים הרבה יותר באימון
- מטפלים טוב יותר ברצפים ארוכים מאוד (+1000 צעדים)
- דורשים יותר נתוני אימון

סיכום: LSTM פתר את בעיית הגרדיאנטים הנעלמים והפך למהפכה בעיבוד רצפים. למרות ש-Transformers שולטים כיום ב-NLP, LSTM עדיין רלוונטי מאוד בסדרות זמן, אותות, ובמשימות שדורשות זיכרון סדרתי.

6 נספח: סיכום משוואות LSTM :Appendix: LSTM Equations Summary

6.1 כל המשוואות במקום אחד: All Equations in One Place

כניסות בצעד זמן t :

- x_t : כניסה נוכחית

- h_{t-1} : מצב מוסתר קודם

- C_{t-1} : מצב תא קודם

משוואות השערים:

שער שכחה (Forget Gate):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

שער כניסה (Input Gate):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

ערכי מועמד (Candidate Values):

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

שער פלט (Output Gate):

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

עדכון מצב התא:

$$(1) \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

חישוב מצב מוסתר:

$$(2) \quad h_t = o_t \odot \tanh(C_t)$$

6.2 מילון סימונים: Notation Guide

טבלה 2: מילון סימונים: Notation Dictionary

סימון / Symbol	משמעות / Meaning	טווח / Range
σ	Sigmoid: $\frac{1}{1+e^{-z}}$	$(0, 1)$
$\tanh$	Hyperbolic tangent	$(-1, 1)$
$\odot$	כפל איבר-איבר / Element-wise multiplication	-
$[h, x]$	שרשור וקטורים / Vector concatenation	-
W	מטריצת משקלים / Weight matrix	$\mathbb{R}$
b	וקטור הטיה / Bias vector	$\mathbb{R}$

6.3 טיפים לכוונון היפר-פרמטרים: Hyperparameter Tuning Tips

1. גודל מצב מוסתר: התחל עם 128 או 256
2. שכבות: 1-3 שכבות LSTM
3. Dropout: 0.2-0.5 בין שכבות
4. קצב למידה: 0.001-0.01
5. אופטימיזר: Adam או RMSprop

6.4 קריאה נוספת: Further Reading

- המאמר המקורי: [2]
- בעיות גרדיאנטים: [1], [6]
- יישומים: [3], [7]
- השוואה עם GRU: [8]
- ספרים: [9], [10]

English References

- 1 Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, 157–166, 1994.
- 2 S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, 1735–1780, 1997.
- 3 A. Graves, "Generating sequences with recurrent neural networks," *arXiv preprint arXiv:1308.0850*, 2013.
- 4 K. Cho et al., "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.
- 5 A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, 5998–6008.
- 6 R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *International Conference on Machine Learning*, PMLR, 2013, 1310–1318.
- 7 I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- 8 J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.
- 9 I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- 10 F. Chollet, *Deep Learning with Python*. Manning Publications, 2017.